



UNINASSAU

Herança

Criação e uso de hierarquia de classes

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

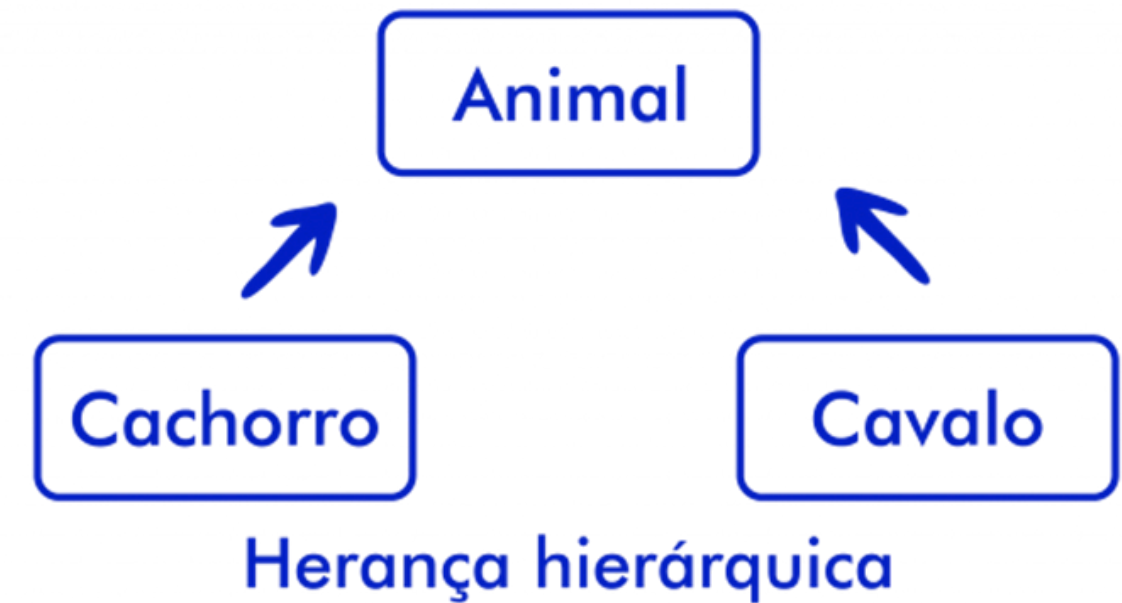
HERANÇA

PILARES DE POO



O QUE É HERANÇA?

- Herança é um mecanismo da POO que permite criar novas classes a partir de classes existentes.
- A nova classe (subclasse) herda atributos e métodos da classe base (superclasse).
- Promove reutilização de código e organização hierárquica.



SINTAXE EM JAVA

```
class Superclasse {  
    // atributos e métodos  
}  
  
class Subclasse extends Superclasse {  
    // novos atributos e métodos  
}
```


EXEMPLO:



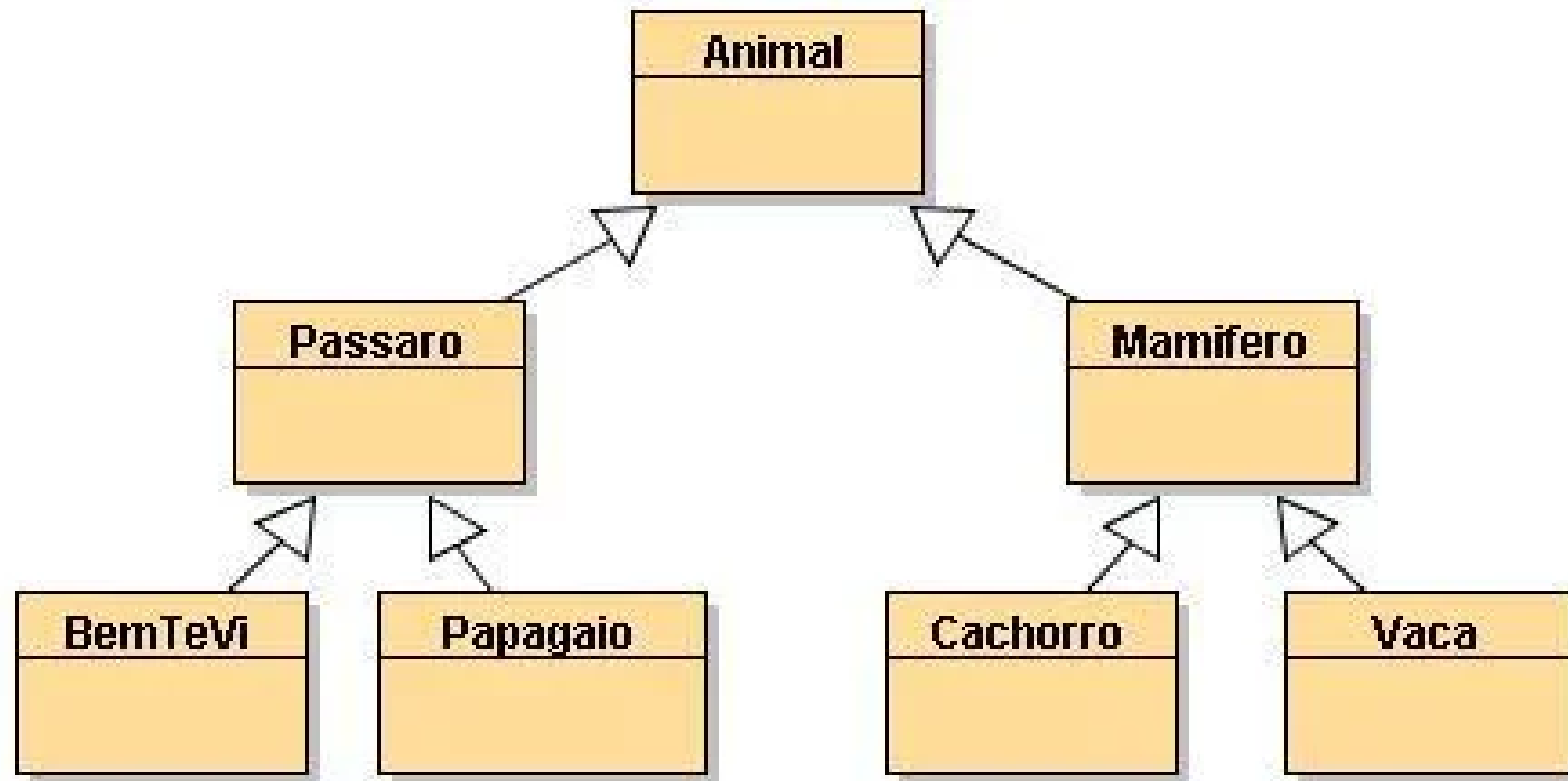
Cachorro herda os **atributos** e **métodos** da **Animal**

```
class Animal {  
    void emitirSom() {  
        System.out.println("Som genérico...");  
    }  
}  
  
class Cachorro extends Animal {  
    void emitirSom() {  
        System.out.println("Latido!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Cachorro();  
        a.emitirSom(); // Saída: Latido!  
    }  
}
```

CRIAÇÃO DE HIERARQUIA DE CLASSES

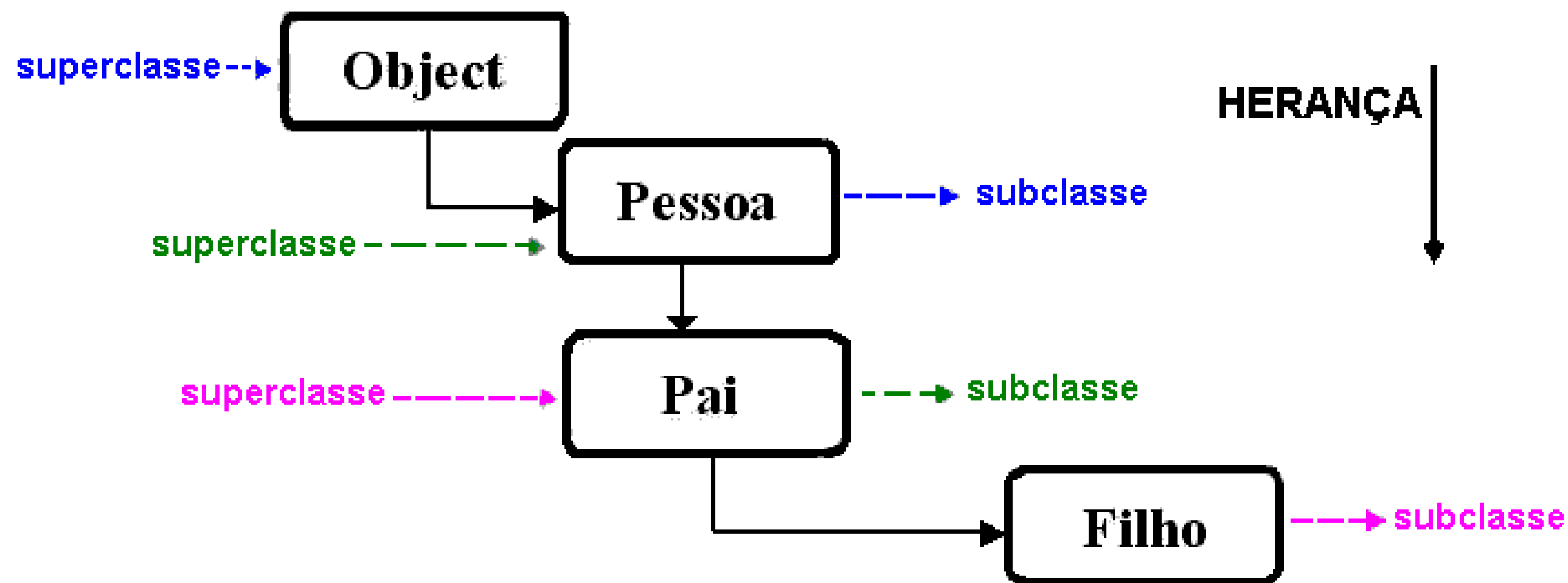
- Uma **hierarquia** organiza classes de forma **genérica** → **específica**.

Exemplo:



BENEFÍCIOS DA HERANÇA

- Reuso de código
- Organização lógica
- Possibilidade de polimorfismo
- Facilita manutenção e extensão do sistema



ARRAYLIST

ARRAYLIST EM JAVA

- É uma **coleção dinâmica** que armazena objetos.
- Substitui o uso de arrays fixos ([]).
- Pertence ao pacote java.util.

Sintaxe básica:

```
import java.util.ArrayList;

ArrayList<String> nomes = new ArrayList<>();
nomes.add("Maria");
nomes.add("João");

System.out.println(nomes.get(0)); // Maria
System.out.println("Tamanho: " + nomes.size()); // 2
```

ARRAYLIST EM JAVA

- Vantagens:
- ✓ Cresce dinamicamente.
- ✓ Possui métodos prontos (**add**, **get**, **size**, **remove**).
- ✓ Facilita percorrer listas com **for-each**.

MÉTODOS PRONTOS DO ARRAYLIST

1) add() → adiciona um elemento à lista.

```
ArrayList<String> nomes = new ArrayList<>();  
nomes.add("Maria");  
nomes.add("João");
```

Agora a lista tem **[Maria, João]**.

2) get(int index) → retorna o elemento que está em uma posição específica.

```
System.out.println(nomes.get(0)); // Maria  
System.out.println(nomes.get(1)); // João
```

MÉTODOS PRONTOS DO ARRAYLIST

3) size() → retorna o número de elementos da lista.

```
System.out.println("Tamanho da lista: " + nomes.size());  
// Saída: 2
```

4) remove() → remove um elemento da lista (pelo índice ou pelo valor).

```
nomes.remove(0); // remove o elemento na posição 0 (Maria)  
nomes.remove("João"); // remove o valor "João"
```


PERCORRER LISTA COM FOR-EACH

- Em vez de usar um **for** com índice (0, 1, 2...), podemos percorrer a lista de forma **mais simples e legível**:

```
for (String nome : nomes) {  
    System.out.println(nome);  
}
```

Esse **for-each** significa:

“Para cada elemento da lista **nomes**, guarde ele temporariamente na variável **nome** e execute o bloco de código.”

RESUMO ARRAYLIST

Resumindo:

add() → colocar item na lista.

get() → acessar item da lista.

size() → descobrir quantos itens tem.

remove() → excluir item.

for-each → jeito fácil de varrer toda a lista sem precisar usar números de posição.

Operador instanceof

DIFERENCIANDO OBJETOS COM INSTANCEOF

- O operador **instanceof** verifica se um objeto é de uma **classe específica**.
- Útil quando trabalhamos com **herança**.
- Evita erros ao converter objetos.

Exemplo:

```
Pessoa p = new Aluno("Maria", 16, "A123");

if (p instanceof Aluno) {
    Aluno a = (Aluno) p; // conversão segura
    a.estudar();
} else if (p instanceof Professor) {
    Professor prof = (Professor) p;
    prof.ensinar();
}
```

Saída esperada:

```
Aluno Maria está estudando.
```

- Use **instanceof** quando precisar descobrir o tipo real de um objeto guardado numa variável de tipo mais genérico

EXEMPLOS

EXEMPLO 1: FUNCIONÁRIOS

Crie uma hierarquia de classes em Java onde:

- A classe **Funcionario** possui **nome** e **salario**.
- A classe **Gerente** herda de **Funcionario** e adiciona o atributo **setor**.
- Crie um método **mostrarDados()** que exiba as informações.
- Teste no método **main**.



EXEMPLO 1: FUNCIONÁRIOS

Passo 1:

- A classe **Funcionario** possui **nome** e **salario**.
- A classe **Gerente** herda de **Funcionario** e adiciona o atributo **setor**.
- Crie um método **mostrarDados()** que exiba as informações.
- Teste no método **main**.

```
class Funcionario {  
    String nome;  
    double salario;  
  
    void mostrarDados() {  
        System.out.println(nome + " - R$ " + salario);  
    }  
}  
  
class Gerente extends Funcionario {  
    String setor;  
  
    void mostrarDados() {  
        System.out.println(nome + " - " + setor + " - R$ " + salario);  
    }  
}
```

EXEMPLO 1: FUNCIONÁRIOS

Passo 2:

- Teste no método **main**.

```
public class Main {  
    public static void main(String[] args) {  
        Gerente g = new Gerente();  
        g.nome = "Ana";  
        g.salario = 5000;  
        g.setor = "TI";  
        g.mostrarDados();  
    }  
}
```

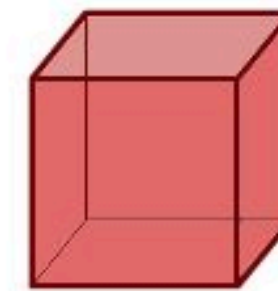

EXEMPLO 2: FORMAS GEOMÉTRICAS

Implemente uma hierarquia de classes em Java para cálculo de áreas:

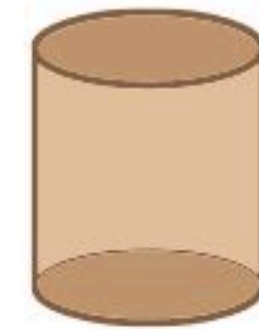
- A classe **Forma** deve ter o método **calcularArea()**.
- A classe **Retangulo** deve herdar de **Forma** e calcular **área** como **largura * altura**.
- A classe **Circulo** deve herdar de **Forma** e calcular **área** como $\pi * \text{raio}^2$.
- Teste no **main** criando diferentes formas.



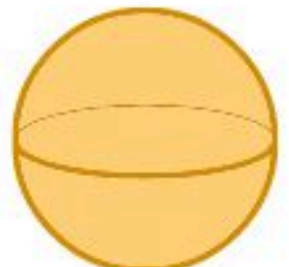
Quadrado



Cubo



Cilindro



Esfera



Círculo



Prisma
Hexagonal



Cone



Pirâmide

EXEMPLO 2: FORMAS GEOMÉTRICAS

Passo 1:

- A classe **Forma** deve ter o método **calcularArea()**.
- A classe **Retangulo** deve herdar de **Forma** e calcular **área** como **largura * altura**.
- A classe **Circulo** deve herdar de **Forma** e calcular **área** como $\pi * \text{raio}^2$.

```
class Forma {  
    double calcularArea() { return 0; }  
}  
  
class Retangulo extends Forma {  
    double largura, altura;  
    Retangulo(double l, double a) { largura = l; altura = a; }  
    double calcularArea() { return largura * altura; }  
}  
  
class Circulo extends Forma {  
    double raio;  
    Circulo(double r) { raio = r; }  
    double calcularArea() { return Math.PI * raio * raio; }  
}
```

EXEMPLO 2: FORMAS GEOMÉTRICAS

Passo 2:

- Teste no **main** criando diferentes formas.

```
public class Main {  
    public static void main(String[] args) {  
        Forma f1 = new Retangulo(4, 5);  
        Forma f2 = new Circulo(3);  
        System.out.println(f1.calcularArea());  
        System.out.println(f2.calcularArea());  
    }  
}
```


EXEMPLO 3: VEÍCULOS

Implemente uma hierarquia de classes em Java para representar veículos:

- A classe **Veiculo** deve ter o atributo **modelo** e o método **mover()**.
- A classe **Carro** deve herdar de **Veiculo** e sobrescrever o método **mover()** exibindo “**O carro anda na estrada**”.
- A classe **Aviao** deve herdar de **Veiculo** e sobrescrever o método **mover()** exibindo “**O avião voa no céu**”.
- No **main**, crie objetos e teste.



EXEMPLO 3: VEÍCULOS

Passo 1:

- A classe **Veiculo** deve ter o atributo **modelo** e o método **mover()**.
- A classe **Carro** deve herdar de **Veiculo** e sobrescrever o método **mover()** exibindo **“O carro anda na estrada”**.
- A classe **Aviao** deve herdar de **Veiculo** e sobrescrever o método **mover()** exibindo **“O avião voa no céu”**.

```
class Veiculo {
    String modelo;
    void mover() {
        System.out.println("O veículo está em movimento.");
    }
}

class Carro extends Veiculo {
    void mover() {
        System.out.println("O carro anda na estrada.");
    }
}

class Aviao extends Veiculo {
    void mover() {
        System.out.println("O avião voa no céu.");
    }
}
```

EXEMPLO 3: VEÍCULOS

Passo 2:

- Teste no método **main**.

```
public class Main {  
    public static void main(String[] args) {  
        Veiculo v1 = new Carro();  
        Veiculo v2 = new Aviao();  
        v1.mover();  
        v2.mover();  
    }  
}
```

Exercícios Propostos

EXERCÍCIO 1: BANCO

Crie uma classe **Conta** com saldo. Crie **ContaCorrente** e **ContaPoupanca** que herdam de **Conta**.
Implemente métodos específicos como **aplicarJuros()** na poupança.



EXERCÍCIO 2: ANIMAIS

Crie uma hierarquia de classes **Animal** → **Mamífero** → **Humano**.

Implemente métodos como **emitirSom()** e **andar()**.

EXERCÍCIO 3: ESCOLA

Crie uma hierarquia de classes que represente pessoas em uma escola:

1. Classe **Pessoa**:

- Atributos: **nome** e **idade**.
- Método: **mostrarDados()** que exibe essas informações.

2. Classe **Aluno** (herda de Pessoa):

- Atributo extra: **matricula**.
- Método: **estudar()** que imprime: "**Aluno <nome> está estudando.**"

3. Classe **Professor** (herda de Pessoa):

- Atributo extra: **disciplina**.
- Método: **ensinar()** que imprime: "**Professor <nome> está ensinando <disciplina>.**"

4. Classe **Escola**:

- Crie o método **main()** para:
- Instanciar 1 aluno e 1 professor.
- Mostrar os dados de cada um.
- Chamar os métodos específicos (**estudar()** e **ensinar()**).

EXERCÍCIO 3: ESCOLA

Dicas extras:

- Reutilize o método **mostrarDados()** da classe **Pessoa** usando **super.mostrarDados()** em **Aluno** e **Professor**.
- Experimente criar uma **lista de pessoas** (**ArrayList<Pessoa>**) e **adicionar tanto alunos quanto professores**.
- Depois, percorra a lista chamando **mostrarDados()** para todos.

REFERÊNCIAS

1. BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4. ed. São Paulo: Pearson Prentice Hall, 2008.
2. SANTOS, Rafael. Introdução à Programação Orientada a Objetos usando Java. 2. ed. Rio de Janeiro: Elsevier, 2013. 336 p.
3. SERSON, Roberto Rubinstein. Programação orientada a objetos com Java 6 – curso universitário. 1. ed. Rio de Janeiro: Brasport, jun. 2009. 492 p.
4. PREISS, Bruno R. Estruturas de dados e algoritmos: padrões de projetos orientados a objetos com Java. Rio de Janeiro: Campus, 2001. 566 p.

Blade Runner 2049 (2017)



Em Blade Runner 2049, após os problemas enfrentados com os Nexus 8, uma nova espécie de replicantes é desenvolvida, de forma que seja mais obediente aos humanos. Um deles é K (Ryan Gosling), um blade runner que caça replicantes foragidos para a polícia de Los Angeles. Após encontrar Sapper Morton (Dave Bautista), K descobre um fascinante segredo: a replicante Rachel (Sean Young) teve um filho, mantido em sigilo até então. A possibilidade de que replicantes se reproduzam pode desencadear uma guerra deles com os humanos, o que faz com que a tenente Joshi (Robin Wright), chefe de K, o envie para encontrar e eliminar a criança.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code